Function definition

def get_pdf_probability(dataset,startrange,endrange):

This defines a function called get_pdf_probability.

It accepts 3 arguments:

- dataset → input data
- startrange → starting value
- endrange → ending value

When we call:

get_pdf_probability(dataset["ssc_p"],40,70)

the values are:

dataset = dataset["ssc_p"]

startrange = 40

endrange = 70

Import pyplot

from matplotlib import pyplot

This imports pyplot from Matplotlib.

pyplot is used to create and display graphs.

pyplot.axvline()

to draw vertical lines on the graph.

Import norm

from scipy.stats import norm

This imports norm from scipy.stats

norm is used to create a **Normal Distribution**.

dist = norm(sample_mean,sample_std)

creates a normal distribution using the mean and standard deviation.

Import Seaborn

import seaborn as sns

This imports the Seaborn library.

sns is just a short name for Seaborn.

We use it to create the distribution graph.

Create distribution graph

ax=sns.distplot(dataset,kde=True,kde_kws={'color':'blue'},color='green')

This plots the given dataset as a distribution graph.

dataset

The data that we want to visualize.

dataset["ssc_p"]

is the SSC percentage data.

kde=True

KDE means **Kernel Density Estimation**.

It creates a smooth curve representing the distribution of the data.

kde_kws={'color':'blue'}

This makes the KDE curve blue.

color='green'

This makes the distribution plot green.

Draw starting vertical line

pyplot.axvline(startrange,color='red')

This draws a vertical red line at the startrange.

startrange = 40

it draws a red line at 40.

Draw ending vertical line

pyplot.axvline(endrange,color='red')

This draws another vertical red line at endrange.

endrange = 70

it draws a red line at 70.

So visually, the graph will show the range:

40 ---------- 70

Assign dataset to sample

sample=dataset

This assigns dataset to the variable sample.

So:

sample

↓

dataset

↓

dataset["ssc_p"]

No new data is created here.

Calculate mean

sample_mean=sample.mean()

This calculates the mean (average) of the sample.

if the output is:

Mean = 67.303

then the average SSC percentage is 67.303.

Calculate standard deviation

sample_std=sample.std()

This calculates the standard deviation of the sample.

Standard Deviation = 10.827

Standard deviation tells us how much the values are spread around the mean.

Print mean and standard deviation

print('Mean=%.3f,Standard Deviation=%.3f' % (sample_mean,sample_std))

This prints the mean and standard deviation.

%.3f

It means:

Display the number with 3 digits after the decimal point.

If the output is :67.30291

will be displayed as:

67.303

Output:

Mean=67.303,Standard Deviation=10.827

Define the distribution

dist=norm(sample_mean,sample_std)

This creates a normal distribution using:

Mean = sample_mean

Standard deviation = sample_std

dist=norm(67.303,10.827)

Now dist represents the normal distribution of your data.

one-line for loop

values=[value for value in range(startrange,endrange)]

This is called a **list comprehension**.

It is a short way of writing a for loop.

startrange = 40

endrange = 70

Then Python sees:

values = [value for value in range(40, 70)]

Break it into 3 parts

[value for value in range(40, 70)]

```
 ↑    ↑         ↑
 |    |         |
 |    |         └── Range of values
 |    └── Loop variable
 └── Value to store
```

range(startrange, endrange)

This generates numbers starting from startrange up to **but not including** endrange.

So:

range(40, 70)

produces:

40, 41, 42, 43, ..., 68, 69

for value

This takes each number one by one and temporarily stores it in the variable value.

For example:

First iteration → value = 40

Second iteration → value = 41

Third iteration → value = 42

...

Last iteration → value = 69

First value

The value before for tells Python:

"Put the current value into the new list."

Therefore the final result is:

values = [40, 41, 42, 43, ..., 68, 69]

"Create a list containing every value from startrange to endrange - 1."

second one-line for loop

probabilities=[dist.pdf(value)for value in values]

this is a list comprehension.

It is a short way of writing a for loop.

values = [40, 41, 42, 43, 44]

"For every value in the values list, calculate dist.pdf(value) and store the result in a new list called probabilities."

Break it into parts

[dist.pdf(value) for value in values]

↑ ↑

| |

| └── Loop through values

└── Calculation for each value

values = [40, 41, 42, 43]

It performs:

dist.pdf(40)

dist.pdf(41)

dist.pdf(42)

dist.pdf(43)

Each result is stored in probabilities.

For example, conceptually:

value PDF result

40 → dist.pdf(40) → 0.008

41 → dist.pdf(41) → 0.010

42 → dist.pdf(42) → 0.012

43 → dist.pdf(43) → 0.014

probabilities = [0.008, 0.010, 0.012, 0.014]

The actual numbers will depend on your sample_mean and sample_std.

Calculate total probability

prob=sum(probabilities)

This adds all the values inside probabilities.

prob = sum(probabilities)

means:

probability1

+ probability2

+ probability3

+ ...

= total

The result is stored in:

prob

Print the result

print("The area between range({},{}):{}".format(startrange,endrange,sum(probabilities)))

This prints the range and calculated result.

startrange = 40

endrange = 70

the output will look like:

The area between range(40,70):0.xxxxx

The .format() method inserts the values into {}.

Return the result

return prob

This sends the calculated prob back to wherever the function was called.

Function call

get_pdf_probability(dataset["ssc_p"],40,70)

This calls the function.

The three arguments are:

dataset["ssc_p"] → data

40 → starting range

70 → ending range